flint-lean on a Kubernetes cluster

Give every agent pod its own workspace: **plain local files** checked out of your S3 bucket before the pod starts, published back on a cadence — or when the agent says so. No FUSE, no NFS, no sidecar in your pod, no credential in your pod.

```
agent pod                                  node
└─ your container ──► /workspace  ◄── bind ── flint-sync worker ──► S3
                      plain files,             (one per mounted
                      real POSIX,               volume, in
                      local speed               flint-workers)
```

A workspace reaches a pod as **one `csi:` volume** served by the `s3.csi.chert.us` node driver. The driver checks the pod's ServiceAccount against the workspace, starts a worker on the pod's node that checks the tree out, binds it into the pod before any of its containers start, and drains it back to the bucket when the pod is deleted.

The trade, stated up front: the workspace is a **local copy**. It must fit the node's disk, writers merge at the manifest rather than editing one live tree (two agents editing one file get last-boundary-wins with the loser's bytes preserved), and other readers see the last published boundary rather than your last write. If that is not your shape, see when not to use lean.

Everything below was run end to end on 2026-09-15 against the published charts `flint-lean` 0.11.0 and `flint-s3-csi` 0.3.0 (images 1.54.0), on a kind cluster (Kubernetes 1.36) with MinIO standing in for the bucket — the only change for that was `endpoint:` and the keys. The commands pin `flint-lean` 0.14.9 and `flint-s3-csi` 0.5.7 (images 1.61.0), this release: it carries lean syncer fixes and the same install surface. Commands are copy-pasteable, not illustrative.

Before you start

An S3 bucket	must already exist. Versioning is not required. Nothing here creates or deletes buckets.
Kubernetes	a CSI node plugin runs on every node that hosts agent pods (a privileged DaemonSet). Tested on 1.34 and 1.36.
Tools	`kubectl`, `helm` 3.8+ (OCI support)
Credentials	see credentials — the agent container never holds one
Node disk	the tree lives on the node's root filesystem, under the kubelet directory; size it for the workspaces a node hosts

---

1. Install

Two charts: `flint-lean` (the `FlintLeanWorkspace` CRD and its operator) and `flint-s3-csi` (the node driver that mounts workspaces, and the credential broker).

```
kubectl create namespace flint-system

# the keys the operator and the broker use (see Credentials)
kubectl -n flint-system create secret generic s3 \
  --from-literal=AWS_ACCESS_KEY_ID=... \
  --from-literal=AWS_SECRET_ACCESS_KEY=... \
  --from-literal=AWS_REGION=us-west-1

helm install flint-lean \
  oci://registry-1.docker.io/dilipdalton/flint-lean --version 0.14.9 \
  -n flint-system \
  --set operatorCredentialsSecret=s3
  # add --set endpoint=http://... for a non-AWS store

helm install flint-s3-csi \
  oci://registry-1.docker.io/dilipdalton/flint-s3-csi --version 0.5.7 \
  -n flint-system \
  --set broker.static.secretRef=s3 \
  --set node.region=us-west-1
```

Check it came up:

```
kubectl get csidriver s3.csi.chert.us
kubectl -n flint-system get pods
# flint-lean-6f45c4486b-tbmkp        1/1   Running
# flint-s3-broker-7dd5fdf688-2k6jm   1/1   Running
# flint-s3-broker-7dd5fdf688-gg2z4   1/1   Running
# flint-s3-csi-node-m2m97            3/3   Running     ← one per node

kubectl -n flint-system logs deploy/flint-lean | tail -1
# flint-lean-operator: watching FlintLeanWorkspace
```

It pulls these images and nothing else:

IMAGE	PULLED BY	WHAT RUNS
`dilipdalton/flint-lean-operator`	the `flint-lean` deployment	the operator
`dilipdalton/flint-s3-csi`	the node DaemonSet and the broker	the CSI node plugin, `flint-s3-broker`
`dilipdalton/flint-s3-worker-lean`	one worker pod per mounted workspace, in `flint-workers`	`flint-sync`, the syncer

flint-lean does not install or require flint-lite — no NFS hub, no `FlintShare` CRD.

> **If `helm install` fails with `permission denied` on `~/Library/Caches/helm`**, something ran `helm` as root with your `HOME` and took ownership of your cache. Either `sudo chown -R "$USER" ~/Library/Caches/helm`, or run with a private cache: `HELM_CACHE_HOME=/tmp/helmcache helm install …` (this guide's own run hit it).

Credentials

Three identities, and **the agent container holds none of them:**

- **The operator** does bucket-admin work (posture probes, a stale-upload sweep) under `operatorCredentialsSecret`, or the pod's ambient chain (IRSA and friends) when that is empty.
- **The broker** (`flint-s3-broker`) hands each mounted workspace's worker a short-lived key, after a TokenReview of the pod's own kubelet-issued token and a check that its ServiceAccount is a consumer of the workspace. The key goes to the worker pod, never the agent pod.
- **The broker's backend** decides what that key is. `static` (the chart default, used above) hands out the one key in `broker.static.secretRef` to every entitled pod. `sts` exchanges the pod's token at an STS for a role's session (`broker.sts.url`, `broker.sts.roleArn`), and `rest` asks your own credential service (`broker.rest.url`). Move off `static` when a per-project credential source exists.

Secret keys are named `AWS_*` **verbatim**.

---

2. Create a workspace

One CR per project subtree, in the namespace your agents run in.

```
kubectl create namespace agents
kubectl -n agents create serviceaccount agent
```

```
apiVersion: chert.us/v1alpha1
kind: FlintLeanWorkspace
metadata:
  name: proj1
  namespace: agents
spec:
  projectId: team-a/proj1        # durable claim identity
  bucket: my-bucket
  keyPrefix: tenants/proj1       # the subtree this workspace owns
  region: us-west-1
  floorSecs: 60                  # publish cadence — and the RPO
  uid: 1000                      # the uid your agent runs as; the syncer runs as it too
  gid: 1000
  consumers:
    serviceAccounts: [agent]     # ABSENT = DENY
```

```
kubectl apply -f workspace.yaml
kubectl -n agents get flintleanworkspace proj1 \
  -o jsonpath='{range .status.conditions[*]}{.type}={.status} {.reason}{"\n"}{end}'
# AccessIsolation=Unknown DecidedByBroker
# SpecAccepted=True Accepted
# SyncerObserved=Unknown NoLiveSyncer
```

You want `SpecAccepted: True`. `SyncerObserved: Unknown` with reason `NoLiveSyncer` is normal before any pod has published. `uid` is required (a workspace refuses to mount without one, and refuses 0): the syncer runs as your app's uid so it can read what the app wrote.

---

3. Mount it in a pod

One volume. That is the whole integration.

```
apiVersion: v1
kind: Pod
metadata:
  name: agent-1
  namespace: agents
spec:
  serviceAccountName: agent            # listed in spec.consumers
  securityContext: { runAsUser: 1000, runAsGroup: 1000, runAsNonRoot: true }
  volumes:
    - name: ws
      csi:
        driver: s3.csi.chert.us
        volumeAttributes: { chert.us/workspace: proj1 }
  containers:
    - name: agent
      image: your-agent:latest
      workingDir: /workspace
      volumeMounts: [{ name: ws, mountPath: /workspace }]
```

```
kubectl -n agents get pod agent-1
# agent-1   1/1   Running            ← your container only
kubectl -n flint-workers get pods
# s3w-cdc625c9ff3a2008   1/1   Running   ← its worker, on the same node
kubectl -n agents exec agent-1 -- ls /workspace/.flint
# AGENTS.md  capabilities.json
```

The checkout completes **before** any container of the pod, init containers included, starts, so your first line of code sees a complete tree. If the workspace is missing or refused, or the ServiceAccount is not a consumer, the pod stays `ContainerCreating` with a `FailedMount` event naming the reason — it never starts against an empty directory. `.flint/AGENTS.md` is the contract for an agent working in the tree.

Read-only agents

An agent that should see the workspace and never change it mounts with `readOnly: true`. List its ServiceAccount under `readOnlyServiceAccounts`:

```
  consumers:
    serviceAccounts: [agent]
    readOnlyServiceAccounts: [viewer]
```

```
      csi:
        driver: s3.csi.chert.us
        readOnly: true
        volumeAttributes: { chert.us/workspace: proj1 }
```

```
kubectl -n agents exec viewer-1 -- sh -c 'echo x > /workspace/x.txt'
# sh: can't create /workspace/x.txt: Read-only file system
kubectl -n agents exec viewer-1 -- grep access /workspace/.flint/capabilities.json
#   "access": "read",
```

Its syncer follows every writer's boundary and publishes nothing, and a `publish` touch is answered `refused-read-only`. A pod on a read-only ServiceAccount **without** `readOnly: true` is refused, naming the fix:

```
FailedMount: ... PermissionDenied desc = ServiceAccount agents/viewer may mount
FlintLeanWorkspace agents/proj1 read-only only (spec.consumers.readOnlyServiceAccounts):
set `readOnly: true` on the pod's csi volume ...
```

What holds it to reads at the bucket depends on the broker backend. `sts` attaches a session policy of reads on the workspace's prefix; `rest` tells your door `"access": "read"`; `static` hands out `broker.static.readSecretRef`'s key if you set one. A `static` broker without a read key says so:

```
kubectl -n flint-system run st --rm -i --restart=Never --image=busybox:1.36 -- \
  wget -qO- http://flint-s3-broker.flint-system.svc/v1/status
# {"backend":"static", ..., "readEnforcement":"cooperative", ...}
```

`cooperative` means the mount and the syncer keep the pod read-only and its key could still write. One workspace carries read-write and read-only agents at once. Set `chert.us/on-behalf-of: <user>` in `volumeAttributes` to name the signed-in user on the broker's audit lines.

---

4. Publish on demand (optional, recommended)

Cadence alone forces a bad choice: publish often and waste work, or publish rarely and let a reader see a half-finished tree. Instead, let the agent declare a coherent point when it finishes a unit of work:

```
# in the agent container
echo hello > /workspace/notes.txt
printf '{"nonce":"task-42"}' > /workspace/.flint/publish

# wait for the answer
until grep -qs task-42 /workspace/.flint/publish.ack; do sleep 1; done
cat /workspace/.flint/publish.ack
# {
#   "status": "ok",
#   "nonces": [ "task-42" ],
#   "seq": 1,
#   "manifest_etag": "\"b2acc362e33c55afed3ef7a1b6516edd\"",
#   "boundary": "sentinel",
#   "completed_unix": 1789456706,
#   "report": { "uploaded": 1, "deleted": 0, "parked": 0, ... }
# }
```

`status: "ok"` means **the named boundary is in the bucket** — not queued, not scheduled — and it carries your change. `partial` means the boundary landed without some path, named in `report.dropped`: an upload that changed under the syncer, which the next boundary publishes, or a path another writer or the UI deleted after your boundary carried it, where their delete stands (write the file again to bring it back). Your delete over their newer version is not dropped: it lands, and their version is kept under a conflict record.

`partial` also means something is **held**, and a held path stays held until it is fixed. The syncer runs as the workspace's uid, so a file it may not read (`chmod 000`, or another uid's file with no read bit for it), a directory it may not list, and a name that is not UTF-8 (a Latin-1 `caf\xe9.txt`, shown with U+FFFD) are held: nothing at or under them is published or deleted, the bucket keeps their last published version, and the rest of the tree publishes as usual. `report.dropped` names them, `.flint-sync/gauges.json` says `withheld_reason: "unreadable"`, and the syncer's last boundary when the pod stops does not attest the tree. Make it readable to the workspace's uid, or rename it, and the next boundary publishes it. (Before 1.61.0 any one of them stopped every publish in the workspace.) A `\` is an ordinary character in a name: `a\b` is one file, published as `a\b`. A `chmod` on its own is published.

A boundary that lost its turn at the publish fence is retried by the cadence, so an ack is late rather than refused. `uploaded: 0` on an `ok` just means nothing had changed since the last boundary.

Costs nothing when unused: measured at 20 bucket requests per 22 s idle with the verbs off, and 20 with them on.

---

5. Verify

```
aws s3 ls s3://my-bucket/tenants/proj1/ --recursive
# tenants/proj1/.flint/lean/chunks/703f8ee1…   ← manifest entries, write-once
# tenants/proj1/.flint/lean/claim
# tenants/proj1/.flint/lean/current            ← the manifest POINTER
# tenants/proj1/.flint/lean/epoch              ← the writers' publish fence
# tenants/proj1/.flint/lean/inbox              ← "please publish" / "please pull" requests
# tenants/proj1/.flint/lean/retired/…          ← what a commit stopped citing, kept 10 min
# tenants/proj1/files/notes.txt@6f1c…          ← your tree lives here, one object per version
```

The `files/` prefix holds the workspace's bytes, one object per version: every write lands at a fresh `<path>@<flush>` key, and nothing overwrites one. The manifest, not the listing, says which version is the tree. The prefix also holds versions a commit stopped citing, kept 10 minutes for readers still on the previous boundary, and uploads no commit has cited yet. So `aws s3 cp` of a bare path finds nothing, and a plain S3 tool or a passthrough mount pointed at `files/` does not see the tree: read it through a pod that mounts the workspace. `.flint/lean/` is control state. To see which clock installed the current boundary:

```
aws s3api head-object --bucket my-bucket \
  --key tenants/proj1/.flint/lean/current \
  --query 'Metadata."flint-boundary-source"'
# "sentinel" | "cadence" | "quiescence" | "drain" | ...
```

`current` is a few hundred bytes and names the chunks holding the entries, so this HEAD stays cheap however large the project grows.

What it costs

Measured on real S3 (us-west-1, one i4i.large node, 2026-09-12; the whole drill, its controls and its rig are in `lean/e2e/perf/results/door-drill-2026-09-12.md`). Every cell is the range over three reps, never a mean. "Host engine" is the syncer binary alone on the node; "as deployed" is the chart, the `s3.csi.chert.us` node plugin, the worker pod and the tenant pod, timed from `kubectl apply` of the tenant pod to Ready (checkout) or from the `publish` touch to its ack.

	HOST ENGINE	AS DEPLOYED
checkout, 6 × 1 GiB	**24.1–25.0 s**	**118.4–161.1 s (n=2)** (reps 2,3: rep 1 paid a lease lockout the drill itself caused)
checkout, 20,000 × 8 KiB	**5.3–5.9 s**	**22.9–24.9 s**
checkout, 4 GiB + 2,000 × 16 KiB	**19.6–19.9 s**	**92.4–115.1 s**
publish, 6 × 1 GiB	**18.3 s**	**19.0–19.4 s**
publish, 20,000 × 8 KiB	**19.5–20.4 s**	**20.8–22.6 s**
publish, 4 GiB + 2,000 × 16 KiB	**50.5–51.7 s** at `uploadPartParallelism: 1` (the v1.51.0 default); **13.9–14.4 s** at 8, the default since the upload window gained its byte bound (`uploadInflightMb` , 256)	**50.8–50.9 s** (at 1)
idle	~5 bucket requests per tick, per workspace	—

The deployed column carries what a pod pays that the engine does not — scheduling, the worker pod, the lease claim, the loop mount (the drill ran with `workers.quota: true` ; the default tree is a plain directory), the bind — a mostly fixed cost of roughly the small tree's gap. Peak worker RSS at the default read window (128 MiB in flight): 403–437 MiB on the 1 GiB objects, 229 MiB on the mixed tree, 72 MiB on the small one, under the worker's 1Gi limit.

The file count binds before the byte count — the manifest is exactly linear in entries, which is why the v1 cap is ~250k files. For scale, the loopback floors: 100k files check out in 49.5 s (first publish 65 s, idle tick 1.85 s); 1M files in 7 m 05 s, with a 264 MiB manifest at ~277 B/entry. Those floors (2026-08-25) predate chunked manifests and 1.61.0's latency work. On 1.61.0's shipped build against an in-memory store (8 cores, 2026-10-24), an idle tick and a one-file publish take about 0.2 s at 200k files and 1 s at 1M; with every request paying 25 ms, a checkout takes about 45 s at 200k files of 8 KiB and 225 s at 1M of 4 KiB, one GET per file with 128 in flight. Against real S3 the request rate (AWS documents 5,500 GET/s per prefix) bounds a checkout the same way. Each lever's numbers are in the CHANGELOG and in `docs/plans/flint-lean-latency-plan-2026-10-23.md` .

Conflict copies, and a lifecycle rule to expire them

When two writers, or a writer and the UI, change one file between boundaries, the boundary that lands second keeps its own bytes and first copies the other version to `<keyPrefix>/.flint/lean/conflicts/<id>/<path>`. The publish ack names the copy. Nothing in lean reads that prefix again: no manifest cites a copy, and checkout, sync and the collector never touch it. So nothing collects it either, and a workload that contends hard grows it (a six-writer drill over 30 shared paths left 4,985 copies in 300 s).

Expire them with a bucket lifecycle rule on that prefix. Deleting a copy cannot break a workspace; it only ends the window in which a person can recover the losing version, so pick a retention long enough to act on a conflict. The rule belongs to the bucket: set it once, however many clusters mount workspaces from it. Nothing runs in any cluster.

```
# A lifecycle write REPLACES the bucket's whole configuration, so start
# from what is there (the flint-lean operator installs an abort rule for
# stale multipart uploads; your own rules may be there too).
aws s3api get-bucket-lifecycle-configuration --bucket my-bucket > lc.json \
  || echo '{"Rules":[]}' > lc.json

# Append one rule per workspace prefix (30 days here), then write it back.
python3 - lc.json tenants/proj1 30 <<'PY'
import json, sys
path, prefix, days = sys.argv[1], sys.argv[2], int(sys.argv[3])
lc = json.load(open(path))
lc["Rules"].append({
    "ID": f"flint-lean-conflicts-{prefix.replace('/', '-')}",
    "Status": "Enabled",
    "Filter": {"Prefix": f"{prefix}/.flint/lean/conflicts/"},
    "Expiration": {"Days": days},
    "NoncurrentVersionExpiration": {"NoncurrentDays": 1},
})
json.dump(lc, open(path, "w"), indent=1)
PY
aws s3api put-bucket-lifecycle-configuration --bucket my-bucket \
  --lifecycle-configuration file://lc.json
```

The filter is a prefix, not a pattern, so it is one rule per workspace, and S3 allows 1,000 rules per bucket. **Never shorten the filter to a common parent** such as `tenants/`: that rule expires every workspace's files, not only its conflict copies. Past the rule limit, use one bucket per group of workspaces. `NoncurrentVersionExpiration` matters only on a versioned bucket, where an expired copy otherwise lingers as a noncurrent version.

When not to use lean

YOU NEED	USE
a tree too large for the pod's disk, or lazy access to a slice of a huge dataset	FUSE (`docs/flint-fuse-architecture.html`)
two pods writing one tree and seeing each other live; byte-range locks; shared sqlite/git across pods	the hub (flint-lite)
readers that cannot tolerate seeing the last boundary instead of the last write	the hub

Choosing lean does not strand the data, but its prefix is not a plain tree: another front end or a passthrough mount pointed at it sees every version under its own key, and none of them reads the manifest. To move a workspace to the hub or to FUSE, mount it in a pod and copy the tree out.

For agents collaborating on code, do **not** reach for a git-over-S3 remote: use plain `git` against a real git host and let flint-lean keep the workspace durable. See `docs/flint-lean-git-workflow.md`.

Tearing down

```
kubectl -n agents delete pod agent-1
kubectl -n agents delete flintleanworkspace proj1   # leaves bucket data intact
helm uninstall flint-s3-csi -n flint-system          # keeps the flint-workers namespace
helm uninstall flint-lean -n flint-system
```

Deleting a pod drains its workspace first; deleting the CR does not delete your data. The bucket subtree is the durable artifact; a new workspace pointed at the same `keyPrefix` picks it up.